

# SYSBENCH MYSQL WORKLOAD TESTING GUIDE

## WHAT IS SYSBENCH ?

---

Sysbench is an open-source benchmarking and load-testing tool.

It is commonly used to test:

- MySQL database performance

- CPU performance

- Memory performance

- Disk I/O performance

- Transaction throughput

- Read/write workloads

- Concurrent database connections

- Query latency

For MySQL tuning, Sysbench is mainly useful for generating a controlled database workload and measuring performance before and after a change.

## WHY DO WE USE SYSBENCH ?

---

Sysbench is useful when we need to answer questions such as:

- How many transactions per second can the server handle?

- How many queries per second can MySQL process?

- What is the average query latency?

- How does MySQL behave with 10, 50, or 100 concurrent connections?

- Did a configuration change improve performance?

- Did an index improve performance?

Does increasing concurrency improve or reduce performance?

Can the server handle the expected workload?

The most important concept is:

TEST -> CHANGE -> TEST AGAIN -> COMPARE

Example:

Before tuning:

TPS: 1,200  
QPS: 5,000  
Avg latency: 85 ms

After adding an appropriate index:

TPS: 1,850  
QPS: 7,200  
Avg latency: 52 ms

This gives measurable evidence instead of relying on assumptions.

## IMPORTANT WARNING

---

DO NOT run a Sysbench benchmark directly against a production database unless the workload has been carefully planned and approved.

Sysbench can generate significant:

CPU usage

Memory usage

Disk I/O

Database connections

Transactions

Queries

A poorly configured benchmark can affect production performance.

Recommended approach:

```
Production MySQL
|
| Collect workload information
```

```
      v
Test/Staging MySQL
      |
      | Run Sysbench
      v
Baseline performance
      |
      | Make tuning/index change
      v
Run same workload
      |
      v
Compare results
      |
      v
Production implementation
```

## 4. SYSBENCH INSTALLATION

Ubuntu/Debian:

```
sudo apt update
sudo apt install sysbench
```

Check the installed version:

```
sysbench --version
```

Example:

```
sysbench 1.0.x
```

## 5. CREATE A TEST DATABASE

Do not use the production database for initial testing.

Create a separate database:

```
CREATE DATABASE sysbench_test;
```

## 6. CREATE SYSBENCH USER

Create a dedicated MySQL user:

```
CREATE USER 'sysbench'@'localhost'
IDENTIFIED BY 'StrongPasswordHere';
```

Grant permissions:

```
GRANT ALL PRIVILEGES
ON sysbench_test.*
TO 'sysbench'@'localhost';
```

Apply privileges:

```
FLUSH PRIVILEGES;
```

For remote testing, the host portion may need to be changed depending on the test architecture.

## TEST CONNECTION

---

Test that the Sysbench user can connect:

```
mysql -u sysbench -p -h 127.0.0.1 sysbench_test
```

If the connection works, exit MySQL:

```
exit;
```

## 8. SYSBENCH MYSQL OLTP WORKLOAD

Sysbench provides several workloads.

For MySQL database testing, one commonly used workload is:

```
oltp_read_write
```

This represents a mixed transactional workload containing reads and writes.

It is useful for testing general OLTP database performance.

## PREPARE TEST DATA

---

Before running the workload, Sysbench needs test tables and data.

Example:

```
sysbench oltp_read_write \  
  --mysql-host=127.0.0.1 \  
  --mysql-port=3306 \  
  --mysql-user=sysbench \  
  --mysql-password='StrongPasswordHere' \  
  --mysql-db=sysbench_test \  
  --tables=10 \  
  --table-size=100000 \  
  prepare
```

Explanation:

```
--mysql-host
```

```
MySQL server hostname or IP address.

--mysql-port
MySQL port. Normally 3306.

--mysql-user
MySQL username.

--mysql-password
MySQL password.

--mysql-db
Test database.

--tables=10
Create 10 Sysbench tables.

--table-size=100000
Insert approximately 100,000 rows per table.

prepare
Creates and populates the test tables.
```

## 10. VERIFY TEST TABLES

Connect to MySQL:

```
mysql -u sysbench -p sysbench_test
```

Run:

```
SHOW TABLES;
```

You should see tables such as:

```
sbtest1
sbtest2
sbtest3
...
sbtest10
```

Check row count:

```
SELECT COUNT(*) FROM sbtest1;
```

Exit:

```
exit;
```

## 11. RUN A BASIC WORKLOAD

Run the workload for 60 seconds with 10 concurrent threads:

```
sysbench oltp_read_write \
```

```
--mysql-host=127.0.0.1 \  
--mysql-port=3306 \  
--mysql-user=sysbench \  
--mysql-password='StrongPasswordHere' \  
--mysql-db=sysbench_test \  
--tables=10 \  
--table-size=100000 \  
--threads=10 \  
--time=60 \  
run
```

### Important options:

```
--threads=10  
    Number of concurrent Sysbench workers.  
  
--time=60  
    Run the test for 60 seconds.  
  
run  
    Execute the workload.
```

## 12. EXAMPLE RESULT

A Sysbench result may contain information similar to:

```
transactions:  
    transactions:          15000  
    (250.00 per sec.)  
  
queries:  
    queries:              300000  
    (5000.00 per sec.)  
  
latency:  
    avg:                  3.98ms  
    max:                  25.31ms
```

The exact output depends on the Sysbench version, workload, configuration, and system.

### IMPORTANT SYSBENCH METRICS

---

## TPS

TPS means Transactions Per Second.

Example:

250 transactions/sec

Higher TPS generally means the system can process more transactions under the tested workload.

However, TPS should always be considered together with latency, CPU, I/O, and other system metrics.

## QPS

QPS means Queries Per Second.

Example:

```
5,000 queries/sec
```

It represents the number of queries processed by the workload.

## LATENCY

Latency represents how long operations take.

Example:

```
Average latency: 4 ms
```

Lower latency is generally better.

Pay attention to:

```
Average latency  
95th percentile latency  
99th percentile latency  
Maximum latency
```

Tail latency is particularly important for applications where slow requests affect users.

## THREADS

Threads represent concurrent workload.

Example tests:

```
1 thread  
10 threads  
25 threads  
50 threads  
100 threads
```

Increasing threads does not always increase performance.

At some point the database or server may become saturated.

## RUN DIFFERENT CONCURRENCY LEVELS

---

To understand how MySQL behaves under different workloads, test several thread counts.

Example:

```
--threads=1
--threads=10
--threads=25
--threads=50
--threads=100
```

Example:

```
sysbench oltp_read_write \
  --mysql-host=127.0.0.1 \
  --mysql-port=3306 \
  --mysql-user=sysbench \
  --mysql-password='StrongPasswordHere' \
  --mysql-db=sysbench_test \
  --tables=10 \
  --table-size=100000 \
  --threads=50 \
  --time=60 \
  run
```

Record the result for each concurrency level.

## EXAMPLE PERFORMANCE TEST TABLE

---

Create a test record like:

| Threads | TPS   | QPS    | Avg Latency |
|---------|-------|--------|-------------|
| -----   | ----- | -----  | -----       |
| 1       | 300   | 6,000  | 3.2 ms      |
| 10      | 1,200 | 24,000 | 8.1 ms      |
| 25      | 1,650 | 33,000 | 15.2 ms     |
| 50      | 1,700 | 34,000 | 29.4 ms     |
| 100     | 1,450 | 29,000 | 68.5 ms     |

This helps identify the point where additional concurrency starts causing contention.

## IMPORTANT: RUN A WARM-UP

---



For more reliable testing, don't immediately use the first few seconds as your final measurement.

A typical approach is:

```
Warm-up period
  |
  v
Actual measurement
  |
  v
Record results
```

Example:

```
--time=300
```

Then discard the initial warm-up period when analyzing results if your test methodology requires it.

For serious benchmarking, run multiple iterations and compare consistent results.

## BEFORE AND AFTER INDEX TESTING

---

Sysbench can be useful when validating an index change.

Example scenario:

Query:

```
SELECT *
FROM orders
WHERE customer_id = 10001;
```

First collect the execution plan:

```
EXPLAIN
SELECT *
FROM orders
WHERE customer_id = 10001;
```

On supported MySQL versions, also consider:

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE customer_id = 10001;
```

## 18. BASELINE TEST

Run the workload BEFORE making the index change.

Record:

TPS  
QPS  
Average latency  
95th percentile latency  
99th percentile latency  
Maximum latency  
CPU usage  
Disk I/O  
Memory usage

Example:

BEFORE INDEX

|              |        |
|--------------|--------|
| TPS:         | 1,200  |
| QPS:         | 24,000 |
| Avg latency: | 8.1 ms |
| 95% latency: | 15 ms  |
| CPU:         | 70%    |

## 19. CREATE THE INDEX

Only create an index after analyzing the actual query and existing indexes.

Example:

```
CREATE INDEX idx_orders_customer_id  
ON orders(customer_id);
```

## 20. RUN THE SAME TEST AGAIN

Run exactly the same Sysbench workload.

Do NOT change multiple variables at the same time.

For example, keep these consistent:

Number of tables  
Table size  
Number of threads  
Test duration  
MySQL configuration  
Server hardware  
Workload type

## 21. COMPARE RESULTS

Example:

| Metric      | Before | After  |
|-------------|--------|--------|
| -----       | -----  | -----  |
| TPS         | 1,200  | 1,850  |
| QPS         | 24,000 | 37,000 |
| Avg latency | 8.1 ms | 5.2 ms |
| 95% latency | 15 ms  | 9 ms   |
| CPU         | 70%    | 65%    |

If the results are consistently better, the change may have improved the workload.

However, validate the actual application queries as well.

### **VERY IMPORTANT: SYSBENCH DOES NOT CHOOSE INDEXES**

---

Sysbench does NOT automatically tell you which index should be created.

Index selection should be based on:

```
Slow Query Log
+
Performance Schema
+
Actual application queries
+
EXPLAIN
+
EXPLAIN ANALYZE
+
Existing indexes
+
Table size/cardinality
+
Query workload
|
v
Index recommendation
|
v
Test
|
v
Validate
```

## **23. CHECK EXISTING INDEXES**

Before creating an index:

```
SHOW INDEX FROM your_database.your_table;
```

Or:

```

SELECT
    TABLE_SCHEMA,
    TABLE_NAME,
    INDEX_NAME,
    COLUMN_NAME,
    SEQ_IN_INDEX,
    CARDINALITY
FROM information_schema.statistics
WHERE TABLE_SCHEMA = 'your_database'
    AND TABLE_NAME = 'your_table'
ORDER BY INDEX_NAME, SEQ_IN_INDEX;

```

## 24. CHECK QUERY EXECUTION PLAN

Example:

```

EXPLAIN
SELECT *
FROM orders
WHERE customer_id = 10001;

```

For MySQL versions supporting it:

```

EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE customer_id = 10001;

```

Look at:

```

access type
possible_keys
key
key_len
rows
filtered
Extra

```

A full table scan is not automatically bad, but for a large table and selective query it may indicate an optimization opportunity.

### **SYSBENCH WORKLOAD TYPES**

---

Common OLTP workloads include:

```

oltp_read_write
    Mixed read/write transactional workload.

oltp_read_only
    Read-only workload.

oltp_write_only
    Write-focused workload.

```

Depending on the Sysbench version and installed scripts, additional workloads may be available.

## **READ-ONLY WORKLOAD**

---

Example:

```
sysbench oltp_read_only \  
  --mysql-host=127.0.0.1 \  
  --mysql-port=3306 \  
  --mysql-user=sysbench \  
  --mysql-password='StrongPasswordHere' \  
  --mysql-db=sysbench_test \  
  --tables=10 \  
  --table-size=100000 \  
  --threads=10 \  
  --time=60 \  
run
```

This is useful when evaluating a read-heavy workload.

## **WRITE WORKLOAD**

---

For a write-oriented workload:

```
sysbench oltp_write_only \  
  --mysql-host=127.0.0.1 \  
  --mysql-port=3306 \  
  --mysql-user=sysbench \  
  --mysql-password='StrongPasswordHere' \  
  --mysql-db=sysbench_test \  
  --tables=10 \  
  --table-size=100000 \  
  --threads=10 \  
  --time=60 \  
run
```

This can help evaluate write performance.

## **CLEAN UP TEST DATA**

---

After testing, Sysbench can remove its test tables:

```
sysbench oltp_read_write \  
  --mysql-host=127.0.0.1 \  
  --mysql-port=3306 \  
  --mysql-user=sysbench \  
  --mysql-password='StrongPasswordHere' \  
  --mysql-db=sysbench_test \  
  --tables=10 \  
run
```

```
--table-size=100000 \  
cleanup
```

Then optionally remove the test database:

```
DROP DATABASE sysbench_test;
```

## 29. BETTER PASSWORD MANAGEMENT

Avoid putting passwords directly into shell command lines in production environments.

Use appropriate MySQL client authentication configuration, environment controls, or another approved secret-management mechanism.

Example MySQL client configuration:

```
[client]  
user=sysbench  
password=StrongPasswordHere  
host=127.0.0.1
```

Protect the file:

```
chmod 600 ~/.my.cnf
```

## 30. MONITOR THE SERVER DURING SYSBENCH

Sysbench results alone are not enough.

Monitor MySQL and operating-system resources.

Check CPU:

```
top
```

or:

```
htop
```

Check memory:

```
free -h
```

Check disk:

```
iostat -xz 1
```

Depending on your environment, also monitor:

```
CPU utilization
```

Memory utilization  
Disk IOPS  
Disk latency  
Network  
MySQL connections  
Buffer pool usage  
Lock waits  
Temporary tables  
Row reads  
Row writes

## 31. PERFORMANCE SCHEMA

MySQL Performance Schema can help identify actual database workload and expensive operations.

For example, inspect statement statistics:

```
SELECT
    DIGEST_TEXT,
    COUNT_STAR,
    SUM_TIMER_WAIT,
    AVG_TIMER_WAIT,
    SUM_ROWS_EXAMINED,
    SUM_ROWS_SENT
FROM performance_schema.events_statements_summary_by_digest
ORDER BY SUM_TIMER_WAIT DESC
LIMIT 20;
```

## 32. SLOW QUERY LOG

For real application tuning, the MySQL slow query log is often more important than a synthetic benchmark.

It can help identify queries that:

Execute slowly  
Run frequently  
Examine many rows  
Consume significant database resources

The actual application workload should be considered when deciding which indexes to create.

### SYNTHETIC VS REAL WORKLOAD

---

Sysbench creates a synthetic workload.

Real applications may behave differently.

For example:

```
Sysbench
|
+-- predictable tables
+-- predictable queries
+-- controlled concurrency
+-- repeatable workload

Real application
|
+-- different queries
+-- different data distribution
+-- different concurrency
+-- caching effects
+-- application-specific behavior
```

Therefore Sysbench should be used as a controlled benchmark, not as the only source of truth.

## RECOMMENDED TESTING PROCESS

---

- Step 1:  
Identify the production workload.
- Step 2:  
Collect slow queries.
- Step 3:  
Review Performance Schema.
- Step 4:  
Check table sizes.
- Step 5:  
Check existing indexes.
- Step 6:  
Run EXPLAIN / EXPLAIN ANALYZE.
- Step 7:  
Create a representative test environment.
- Step 8:  
Load representative data.
- Step 9:  
Run Sysbench baseline.
- Step 10:  
Record TPS, QPS and latency.



Step 11:  
Apply one tuning change.

Step 12:  
Run the same Sysbench workload again.

Step 13:  
Compare results.

Step 14:  
Validate the actual application query.

Step 15:  
Implement the approved change in production.

## EXAMPLE COMPLETE TEST

---

### PREPARE:

```
sysbench oltp_read_write \  
  --mysql-host=127.0.0.1 \  
  --mysql-port=3306 \  
  --mysql-user=sysbench \  
  --mysql-password='StrongPasswordHere' \  
  --mysql-db=sysbench_test \  
  --tables=10 \  
  --table-size=100000 \  
  prepare
```

### BASELINE:

```
sysbench oltp_read_write \  
  --mysql-host=127.0.0.1 \  
  --mysql-port=3306 \  
  --mysql-user=sysbench \  
  --mysql-password='StrongPasswordHere' \  
  --mysql-db=sysbench_test \  
  --tables=10 \  
  --table-size=100000 \  
  --threads=10 \  
  --time=60 \  
  run
```

### RECORD:

TPS:  
QPS:  
Average latency:  
95th percentile latency:  
99th percentile latency:  
Maximum latency:  
CPU:  
Memory:

Disk I/O:

**MAKE ONE CHANGE:**

```
CREATE INDEX idx_orders_customer_id  
ON orders(customer_id);
```

**RUN THE SAME TEST:**

```
sysbench oltp_read_write \  
  --mysql-host=127.0.0.1 \  
  --mysql-port=3306 \  
  --mysql-user=sysbench \  
  --mysql-password='StrongPasswordHere' \  
  --mysql-db=sysbench_test \  
  --tables=10 \  
  --table-size=100000 \  
  --threads=10 \  
  --time=60 \  
run
```

**COMPARE:**

Before TPS:

After TPS:

Before QPS:

After QPS:

Before Avg latency:

After Avg latency:

Before 95% latency:

After 95% latency:

## 36. BEST PRACTICES

Never benchmark production without approval.

Use a dedicated test database.

Use a dedicated Sysbench MySQL user.

Use realistic data volumes.

Use realistic concurrency.

Run multiple test iterations.

Warm up the workload.

Change only one important variable at a time.

Monitor CPU, memory, disk and MySQL while testing.

Record all benchmark results.

Use EXPLAIN/EXPLAIN ANALYZE for query-specific tuning.

Check existing indexes before creating new ones.

Avoid redundant indexes.

Do not assume that higher TPS always means a better system.

Always consider latency and resource utilization.

Validate improvements against the real application workload.

## KEY CONCEPT

---

Sysbench answers:

"How does MySQL perform under this controlled workload?"

EXPLAIN answers:

"How is MySQL executing this particular query?"

Performance Schema / Slow Query Log answers:

"Which queries are actually consuming resources?"

Together:

```
Real workload
+
Slow Query Log
+
Performance Schema
+
EXPLAIN ANALYZE
+
Sysbench
|
v
Evidence-based MySQL tuning
```

## 38. FINAL SUMMARY

Sysbench is a benchmarking and load-generation tool.

For MySQL tuning, use it to:

1. Create a controlled workload.

2. Establish a performance baseline.
3. Apply a tuning change.
4. Run the same workload again.
5. Compare TPS, QPS and latency.
6. Determine whether the change improved performance.

Do not use Sysbench alone to decide which indexes to create.

For index tuning, combine:

Slow Query Log  
Performance Schema  
EXPLAIN  
EXPLAIN ANALYZE  
Existing indexes  
Table statistics  
Real application workload  
Sysbench validation

The goal is not simply to make the benchmark faster.

The goal is to make the database perform better for the actual application workload while maintaining acceptable CPU, memory, disk I/O, concurrency and latency.